COPPER DIGITAL · AWS PARTNER TEAM · SUPPLEMENT

OASIS APIS — Section 0

Dataset Generation Strategy

March 2026 | Addendum to Partner PRD v2.2 | Copper Digital Internal

Covers: Phase A synthetic bootstrap · Phase B HIL feedback · Phase C live production harvest

📋 **PURPOSE**

WHY THIS SECTION EXISTS

The optimizer in this system requires labeled patient records to function — it cannot run on an empty dataset. Before a single optimization round can be executed, the partner team must generate an initial corpus of synthetic patient records that are clinically consistent, structurally valid, and internally coherent across all five document types.

This section describes how to generate that bootstrap dataset and how the dataset evolves over time as real data becomes available from Copper's production system.

SECTION 0 **Dataset Generation Strategy**

Three phases from synthetic bootstrap to live production data

0.1 The Three-Phase Dataset Strategy

The labeled dataset that drives the optimizer evolves across three distinct phases. Each phase feeds into the same optimizer pipeline — only the source and quality of the labeled records changes.

Phase	Source	When	Minimum records	Notes
A — Synthetic Bootstrap	LLM-generated hypothetical patients	Before go-live; immediately available	50 records across 8+ archetypes	No real patient data needed; purely LLM-generated with clinical logic rules
B — Human-in-the-Loop Feedback	Clinician corrections on system-generated OASIS	Once Copper's system is live with real nurses	Add 10–20 records per agency per month	Corrections become the gold standard; replaces synthetic for

				those archetypes over time
C — Production Harvest	Clinician-approved OASIS from Copper's live system	Once system reaches stable accuracy (>85%)	Accumulates automatically	Full-loop: system generates, clinician approves, approved record joins training set

⚠️ **Ownership**

Phase A is the partner team's responsibility and is required before any optimizer work begins. Phases B and C are owned by Copper — the partner team's system must accept records from all three phases in the same format, with no code changes required when the source changes.

SECTION 0A Phase A — Synthetic Bootstrap Dataset

LLM-generated hypothetical patients to seed the optimizer

0A.1 What to Generate

For each synthetic patient, five documents must be generated in a strict dependency order. Each document is built on top of the ones before it. All five must be internally consistent — the same clinical facts must appear across all of them.

Document	Format	Depends on	Always present?
1. Referral packet	Plain text mimicking a hospital discharge referral. Must include: demographics, admission/discharge dates, primary and secondary diagnoses with ICD-10 codes, medication list, physician orders, homebound justification.	Patient archetype metadata	Yes — mandatory
2. Clinical summary	Free-text narrative as if written by the SOC nurse. Covers: current functional status, ADL observations, home environment, vital signs, patient-reported pain and goals. Must reflect what the referral implies but adds direct observation detail.	Referral packet	70% of records — mark absent in 30%
3. Gap questions and answers	Structured JSON in tap_tap_gap_answers.json format. Contains only the OASIS items that could NOT be extracted from the referral or clinical summary. Must include all always-gap items (BIMS, PHQ-2/9, GG discharge goals, wound measurements).	Referral + clinical summary	Yes — mandatory

4. Medication list	Structured JSON with three layers: hospital discharge list, patient pill bottles found at home, patient-reported OTC supplements. Must include deliberate discrepancies — see rules below.	Referral (for discharge meds)	Yes — mandatory
5. Gold-standard OASIS	Flat JSON mapping every OASIS-E1 item code to its correct value. Derived by applying CMS coding rules to the facts stated in documents 1–4. Must produce a valid HIPPS code when run through the CMS PDGM grouper.	All four documents above	Yes — mandatory

0A.2 Generation Workflow — Strict Dependency Order

Each document must be generated in sequence. Do not generate them in parallel — later documents must be consistent with earlier ones. The LLM must be given all prior documents as context when generating each new one.

```
WORKFLOW for each synthetic patient:

  STEP 1 — Define patient archetype metadata
    Choose archetype from the approved list (see 0A.3)
    Set: age, gender, primary diagnosis, comorbidities, functional baseline,
         referral format style, whether clinical summary exists,
         PDGM episode timing (early/late), admission source (hospital/community)

  STEP 2 — Generate referral packet
    INPUT:  patient archetype metadata
    OUTPUT: referral_packet.txt
    RULES:  include all mandatory referral sections (see 0A.4)
            vary format style per archetype metadata (clean EMR / messy fax /
minimal)
            all ICD-10 codes must be valid and match diagnoses in the narrative

  STEP 3 — Generate clinical summary  (skip if metadata says absent)
    INPUT:  referral_packet.txt + archetype metadata
    OUTPUT: ambient_scribe.txt
    RULES:  nurse voice, first person, timestamped observations
            functional findings must be CONSISTENT with but more detailed than
referral
            include: vitals, ADL observations, home safety, patient goals, pain
scale
            do NOT include standardised BIMS or PHQ wording — those go in gap
answers only

  STEP 4 — Generate gap questions and answers
    INPUT:  referral_packet.txt + ambient_scribe.txt + archetype metadata
    OUTPUT: tap_tap_gap_answers.json
    RULES:  identify which OASIS items cannot be extracted from documents 1-2
            always include: BIMS (C0200-C0500), PHQ-2/9 (D0150-D0160),
                            all GG discharge goals, wound measurements if wounds
present
            answers must be consistent with clinical presentation in referral +
summary
```

```
            use tap_tap_gap_answers.json format exactly (see Section 2.2 of main
PRD)

  STEP 5 — Generate medication list
    INPUT:  referral_packet.txt (for discharge med list)
    OUTPUT: medication_list.json
    RULES:  must have all three layers: hospital_discharge, pill_bottles,
patient_reported_otc
            must include the four required discrepancies (see 0A.5)

  STEP 6 — Generate gold-standard OASIS
    INPUT:  all four documents above + CMS OASIS-E1 coding rules
    OUTPUT: oasis_gold_standard.json
    RULES:  apply CMS coding rules to facts in documents 1-4
            all skip patterns must be correctly applied
            GG items must reflect functional status stated in clinical summary
            BIMS and PHQ values must match gap answers exactly
            result must produce a valid HIPPS code

  STEP 7 — Validate consistency
    Cross-check all five documents for contradictions (see 0A.6)
    If inconsistencies found: regenerate the affected document, not all five
```

0A.3 Patient Archetypes — Minimum Coverage

The bootstrap dataset must cover at least 8 archetypes before the optimizer can start. The 50-record minimum should be distributed as shown. Copper's TRAINING_DATA_BRIEF specifies the full 30-archetype target for the long-term dataset — the partner team covers the first 8 to unblock Phase 1.

Archetype	PDGM Group	Min records	Priority
Total Knee Replacement (post-surgical)	MS Rehab	8	High — most common in Copper's current agencies
CHF Exacerbation	MMTA Cardiac	8	High — second most common
Diabetic Foot Ulcer	Wounds	6	High — complex, tests wound coding
CVA / Stroke Rehab	Neuro/Stroke	6	High — tests GG and cognitive sections
Hip Fracture	MS Rehab	5	Medium
COPD Exacerbation	MMTA Respiratory	5	Medium
Sepsis / Cellulitis Recovery	MMTA Infectious	6	Medium — tests antibiotic / IV coding
Patient Refuses or Cannot Answer	Edge Case	6	Critical — tests system's handling of missing data

Additional archetypes (expand to 30 over time)	Various	—	Copper provides priority order

0A.4 Referral Packet — Required Sections

Every synthetic referral must include all of the following sections. Missing sections cause the pipeline to either hallucinate data or flag excessive gap questions — both corrupt the training signal.

Section	What it must contain	Common omission to avoid
Patient demographics	Full name (synthetic), DOB, gender, address, Medicare ID (synthetic), primary insurer	Do not use real Medicare ID formats — use clearly fictional patterns
Admission and discharge dates	Hospital admit date, discharge date, referral date, expected SOC date	Dates must be internally consistent — discharge before SOC
Primary diagnosis	Full ICD-10 code + description matching the archetype	Do not put the ICD code in a different section from the description
Secondary diagnoses	Up to 4 comorbidities with ICD-10 codes	Must be clinically plausible given the primary — no random codes
History and physical (H&P)	Narrative clinical history using real abbreviations (e.g. 78 y/o F w/ h/o CHF)	Narrative must reference the same diagnoses as the code list
Discharge medication list	Full med list: name, dose, route, frequency — used as Layer 1 of medication_list.json	Flag high-risk medications explicitly: anticoagulants, insulin, opioids, digoxin
Physician orders for home health	Disciplines ordered (SN, PT, OT), frequencies, specific skilled nursing procedures	Must match the clinical picture — do not order PT for a cardiac-only patient
Homebound justification	Two-part: (1) condition causing homebound status, (2) effort required to leave home	Both parts required — missing either causes HIPPS validation failure
Face-to-face (F2F) encounter	Physician certification that patient is homebound and needs skilled care	Vary status: 60% present and complete, 20% present but incomplete, 20% missing

0A.5 Medication List — Required Discrepancies

Every synthetic medication list must contain at least these four deliberate discrepancies. They test the system's ability to detect reconciliation issues and code N0415 (high-risk medications) correctly.

Discrepancy Type	Description	Example
Missing at home	One medication from the hospital discharge list is not physically present in the home	Warfarin on discharge list; no warfarin bottle found at home
Wrong dose bottle	One bottle found at home has a different dose than the current discharge prescription	Metoprolol 25mg bottle at home; discharge list says 50mg
OTC supplement not on any official list	Patient reports taking a supplement not listed anywhere in the referral	Patient mentions fish oil 1000mg daily — not on any official list
Dose discrepancy between layers	The dose on a bottle label differs from what the patient reports taking	Lisinopril bottle says 10mg; patient says they take 20mg

```
// medication_list.json structure — three layers required
{
  "hospital_discharge_list": [
    { "name":"Warfarin", "dose":"5mg", "route":"oral", "frequency":"daily",
      "is_high_risk": true }
  ],
  "patient_pill_bottles": [
    { "name":"Metoprolol", "dose":"25mg", "route":"oral", "frequency":"twice
daily",
      "note":"bottle dose differs from discharge prescription of 50mg" }
    // Warfarin absent — required discrepancy 1
  ],
  "patient_reported_otc": [
    { "name":"Fish Oil", "dose":"1000mg", "route":"oral", "frequency":"daily",
      "note":"not on any official list" }
  ]
}
```

0A.6 Cross-Consistency Validation

After generating all five documents for a patient, validate internal consistency before accepting the record. Inconsistencies create false training signal — the optimizer will try to 'fix' a config that is actually correct.

Check	What to verify
Diagnosis consistency	ICD-10 codes in the referral code list match the diagnoses described in the H&P narrative
Functional consistency	GG scores in the gold standard match the functional level described in the clinical summary. If patient is documented as fully dependent in bathing, GG0130A1 must be '01'.
BIMS consistency	C0500 total score in gold standard equals the sum of C0200 + C0300 + C0400 from gap answers
PHQ-2 gate	If PHQ-2 score (D0150A1 + D0150B1) < 3, all PHQ-9 items must be skipped in the gold standard

Wound consistency	If wound is present in referral, M1311 must be non-zero in gold standard, wound measurements must be in gap answers
Medication consistency	High-risk medications in the discharge list must trigger N0415 flag in gold standard
Homebound consistency	Homebound justification in referral must align with functional limitations described in clinical summary
Date consistency	All dates must be internally ordered: hospital admit < discharge < referral date < SOC date

```
VALIDATION PSEUDOCODE (run after generating all 5 documents):

FUNCTION validate_patient_record(patient_record):
  errors = []

  // Check GG scores match clinical summary
  FOR EACH gg_item IN [GG0130A1, GG0130B1, GG0170A1, ...]:
    gold_value  = gold_standard[gg_item]
    gap_answer  = gap_answers.find(field_code = gg_item).answer
    IF gold_value != gap_answer:
      errors.ADD('GG mismatch: {gg_item} gold={gold_value} gap={gap_answer}')

  // Check BIMS total
  bims_total = C0200 + C0300 + C0400 from gap_answers
  IF gold_standard['C0500'] != bims_total:
    errors.ADD('BIMS total mismatch')

  // Check PHQ-2 gate
  phq2_score = D0150A1 + D0150B1 from gap_answers
  IF phq2_score < 3:
    FOR EACH phq9_item IN [D0150C1 through D0150J2]:
      IF gold_standard[phq9_item] is not null/skip:
        errors.ADD('PHQ-9 item should be skipped: {phq9_item}')

  // Check dates
  IF discharge_date <= admit_date OR referral_date < discharge_date OR soc_date <
referral_date:
    errors.ADD('Date ordering violation')

  IF errors is not empty:
    LOG all errors
    RETURN invalid — do NOT add this record to the dataset
  RETURN valid
```

0A.7 LLM Generation Guidance

Use Claude Sonnet for most archetypes. Use a more capable model (Claude Opus) for complex patients — high comorbidity counts, edge cases, patients who refuse assessments. Simpler archetypes (post-surgical, straightforward cardiac) can use Claude Sonnet reliably.

Patient complexity	Recommended model	Reason

Standard archetype (1–2 diagnoses, full cooperation)	Claude Sonnet	Fast, cheap, reliable for well-defined clinical presentations
High comorbidity (4+ conditions: DM + CHF + COPD + CKD)	Claude Opus or equivalent	Requires more complex clinical reasoning to keep all facts consistent
Edge cases (patient refuses, non-English, dual eligible, polypharmacy 20+ meds)	Claude Opus or equivalent	Higher risk of consistency errors; worth the additional cost

Generation prompting strategy

When calling the LLM to generate each document, the prompt must include:

- The patient archetype metadata (diagnosis, age, functional baseline, comorbidities)
- All previously generated documents as context — not just a summary, the full text
- An explicit instruction to be internally consistent with all prior documents
- The specific CMS coding rules relevant to the document being generated (e.g. GG coding conventions when generating the gold standard)
- An explicit instruction to use realistic but entirely synthetic identifiers — fictional names, fictional Medicare IDs, fictional NPIs, fictional addresses

> NO REAL PHI IN GENERATED DATA
> All patient names, Medicare IDs, NPIs, addresses, and physician names must be entirely fictional. Do not use real clinical examples from internet sources. Do not use real person names — generate names that are clearly synthetic.
> Label every generated file with: SYNTHETIC — NOT REAL PATIENT DATA

SECTION 0B Phase B — Human-in-the-Loop Feedback Data

Clinician corrections from Copper's live system as training data

> ℹ️ **Ownership**
> Phase B is owned and operated by Copper — the partner team does not build this.
> The partner team's system must accept Phase B records in the same format as Phase A records.
> No code changes should be required when the dataset source changes from A to B.

0B.1 What Phase B Records Are

Once Copper's production system is live with real home health agencies, every patient assessment goes through a nurse review cycle. When the nurse corrects an LLM-generated

OASIS item, that correction is the ground truth. A Phase B record is assembled from one complete nurse review session:

Document	Source in Phase B	Notes
referral_packet.txt	Real de-identified referral from Copper's system	PHI stripped; synthetic substitutes for name, DOB, Medicare ID, address
ambient_scribe.txt	Real de-identified nurse SOC transcript	PHI stripped; optional — only if ambient capture was active
tap_tap_gap_answers.json	Real gap answers submitted by the nurse through Copper's tap-tap UI	Exact same format as Phase A — no conversion needed
medication_list.json	Real de-identified medication list from Copper's system	PHI stripped
oasis_gold_standard.json	The clinician-corrected and signed OASIS from Copper's system	This is real ground truth — highest quality training signal

0B.2 De-identification Requirement

All Phase B records contain real patient data and must be de-identified before being added to the training dataset. This is Copper's responsibility. The following fields must be replaced with realistic synthetic substitutes:

- Patient name → synthetic name
- Date of birth → adjusted to preserve age bracket only (e.g. keep age 74, not exact DOB)
- Medicare ID / MBI → synthetic ID in valid format
- Home address → synthetic address in the same US state
- Physician names and NPIs → synthetic physician names and fictional NPIs
- Agency name and CCN → replaced with a fictional agency identifier

0B.3 How Phase B Data Replaces Synthetic Data Over Time

As Phase B records accumulate, they gradually replace synthetic records for the same archetype in the training set. The replacement strategy:

```
STRATEGY for blending Phase A and Phase B records:

  IF Phase B records for archetype >= 10:
    USE Phase B records only for that archetype in training set
    RETIRE Phase A records for that archetype

  IF Phase B records for archetype < 10:
    USE all available Phase B records PLUS Phase A records to reach minimum count
    Phase B records get higher weight in accuracy scoring (real > synthetic)

  IF Phase B records for archetype == 0:
```

```
    USE Phase A records only

  // The optimizer does not need to know the source — it processes all records
  // identically. The source field in metadata is for tracking only.
```

SECTION 0C Phase C — Production Harvest

Auto-accumulation from the live system once accuracy is stable

> ℹ️ **Ownership**
> Phase C is also owned by Copper. The partner team's system needs no changes to support it. It is described here so the partner team understands the full data lifecycle and builds the dataset loader to accept a source metadata field without breaking.

0C.1 How Phase C Works

Once the system reaches stable accuracy (> 85% overall, > 90% HIPPS), Copper's production pipeline can begin harvesting approved records automatically. Every time a nurse approves an OASIS without corrections — or with only minor administrative corrections — that record is eligible for Phase C harvest.

```
PHASE C HARVEST LOGIC (runs nightly in Copper's system):

  FOR EACH completed patient assessment in the last 24 hours:
    IF nurse_approval_status == 'APPROVED'
    AND correction_count <= 2          // only minor corrections allowed
    AND all_required_documents_present
    AND hipps_code_valid:
      de_identify(patient_record)
      add_to_training_dataset(patient_record, source='PHASE_C')

  // Phase C records accumulate automatically — no manual curation needed
  // This is the flywheel: the better the system gets, the more training data
  // it generates, which makes it better still
```

0C.2 Full Data Lifecycle Summary

The diagram below shows how the dataset evolves from launch through mature operation:

Time period	Dataset composition	Optimizer input quality	Expected accuracy
Pre-launch (partner team)	100% Phase A synthetic — 50 records, 8 archetypes	Low — synthetic data has consistent but artificial patterns	Baseline; establishes starting config

Early live (months 1–3)	Mostly Phase A; Phase B records trickling in from first agencies	Medium — real ground truth for some archetypes	Improving as Phase B replaces synthetic
Growth (months 4–9)	Phase B dominant for high-volume archetypes; Phase A fills gaps	High — majority is real clinician-corrected data	Should reach 85%+ target
Mature (months 10+)	Phase C dominant; Phase B for new archetypes; Phase A retired	Highest — continuously refreshed real approved data	90%+ sustained; self-reinforcing flywheel

SECTION 0D Dataset Metadata & Record Schema

Tracking record provenance and ensuring format consistency across all phases

0D.1 metadata.json — One Per Patient Record

Every patient record directory must include a metadata.json file. This is used by the optimizer to track dataset composition, filter by archetype or phase, and ensure the train/holdout split is archetype-stratified.

```
// metadata.json — present in every patient_XXXX/ directory
{
  "patient_id":          "SYN_0001",          // SYN_ = synthetic, EMR_ =
de-identified real
  "source":              "PHASE_A",            // PHASE_A | PHASE_B | PHASE_C
  "archetype":           "total_knee_replacement",
  "pdgm_group":          "MS_REHAB",
  "admission_source":    "hospital",           // hospital | community
  "episode_timing":      "early",              // early (0-29 days) | late (30+
days)
  "age_bracket":         "75-84",
  "gender":              "F",
  "comorbidity_count":   3,
  "has_ambient_scribe":  true,
  "has_clinical_note":   false,
  "f2f_status":          "present_complete",   // present_complete |
present_incomplete | missing
  "referral_format":     "clean_emr",          // clean_emr | messy_fax | minimal
  "validation_status":   "passed",             // passed | failed | pending
  "generated_by":        "claude-sonnet",
  "generated_date":      "2026-03-07",
  "clinician_validated": false                 // true only for Phase B and C
records
}
```

0D.2 Train / Holdout Split Rules

The 80/20 split between training and holdout sets must be stratified by archetype — not purely random. This ensures every archetype is represented in the holdout set for validation.

```
FUNCTION split_dataset(all_records, train_ratio=0.80, seed=42):
  // Group records by archetype
  groups = group all_records by metadata.archetype

  train_set   = []
  holdout_set = []

  FOR EACH archetype_group IN groups:
    shuffle archetype_group using seed
    n_holdout = max(1, floor(len(archetype_group) * (1 - train_ratio)))
    holdout_set.ADD  first n_holdout records
    train_set.ADD    remaining records

  // Every archetype must have at least 1 record in holdout
  // Holdout set must NEVER be used during optimization — only for final
validation
  RETURN train_set, holdout_set
```

SECTION 0E Edge Cases — Required Coverage

Scenarios that stress the pipeline and must be in the dataset

The following edge cases must be represented in the Phase A synthetic dataset. They expose weaknesses in the config that standard archetypes would never reveal. Each edge case has a minimum count:

Edge Case	Target count in Phase A	Why it matters
Patient refuses some assessments	4 records	Tests that refused items code correctly as '09' or per CMS rules — not as missing
Caregiver answers instead of patient	4 records	Tests that caregiver-reported functional status is correctly flagged vs. patient-reported
Non-English speaking patient (interpreter needed)	3 records	Tests BIMS and PHQ administration when language barrier is documented
20+ medications (polypharmacy)	4 records	Tests N0415 multi-flag coding and medication reconciliation under complex conditions
Multiple wound sites	3 records	Tests M1311 and wound staging when more than one active wound is present
Minimal / incomplete referral (no labs, no F2F)	4 records	Tests system behaviour when referral is sparse — maximum gap questions expected
Patient refuses or cannot answer BIMS	4 records	Tests the C0100 interview-not-conducted pathway and its downstream skip logic
Readmission within 30 days (late episode)	4 records	Tests late-episode PDGM coding and whether prior episode data is correctly handled

SECTION 0F Phase A Acceptance Checklist

What Copper validates before the optimizer starts

Before the partner team can begin Phase 1 (harness) and Phase 2 (eval engine), Copper's clinical lead must validate a sample of the generated records. This gate cannot be skipped.

Check	Method	Owner
50 records generated across at least 8 archetypes	Count records; verify metadata.json archetype field	Partner team
All mandatory files present in every record directory	Run automated file presence check	Partner team
All records pass the 0A.6 consistency validation	Run validation script; zero failures permitted	Partner team
ICD-10 codes are valid	Validate codes against CMS ICD-10-CM code list	Partner team
Gold standard produces valid HIPPS code for > 90% of records	Run HIPPS grouper tool — Copper provides access	Copper + partner team
BIMS and PHQ values consistent across gap answers and gold standard	Automated cross-check	Partner team
Clinical lead spot-checks 10 records for clinical plausibility	Manual review by Kathy or Michael — 30 mins each	Copper clinical team
All edge case types represented	Check metadata.json archetype distribution	Partner team
No real PHI present	Verify all identifiers are synthetic format	Partner team + Copper

CONTACTS

CONTACTS FOR SECTION 0
Manas (CTO) — generation model access, HIPPS grouper tool, PHI de-identification guidance
Kathy (Clinical Lead) — spot-check review of generated records, clinical accuracy sign-off
Balaji — any S3 storage setup for the dataset, de-identification pipeline for Phase B/C
Questions on OASIS coding rules during gold standard generation: ask Kathy first.

⚠️ Copper Digital Internal / Partner-Restricted. Do not distribute. March 2026.

COPPER DIGITAL · FOR AWS PARTNER TEAM

OASIS Auto Prompt Improvement System

External Partner Product Requirements — v2.2

March 2026 | Copper Digital Internal | Partner-Restricted

Stack: Python 3.11 · AWS Bedrock (Claude Sonnet) · pgvector / ChromaDB · PostgreSQL

📋 **PURPOSE**

PURPOSE OF THIS DOCUMENT
This PRD gives the external AWS partner team everything needed to build the OASIS Auto Prompt
Improvement (OAPI) System. The system improves an LLM config file by running a labeled patient
dataset through a pipeline, measuring accuracy against gold-standard OASIS outputs, and using an
LLM to iteratively refine the config until accuracy plateaus.

All logic described here is pseudocode — the partner team designs and implements their own solution.
Copper will provide: the group config file, the labeled dataset, and clinical reference material.
No Copper production code is shared or called.

⚠️ KEY CONSTRAINT
The partner team builds their own complete pipeline from scratch.
They do NOT import, call, or wrap any Copper codebase module.
The pseudocode in this document describes required behaviour, not implementation.

SECTION 1 **System Overview**

What the system does and the artefact it improves

1.1 What the System Does

The OAPI system takes a labeled dataset — patient intake documents paired with clinician-validated OASIS outputs — and iteratively improves a group config file. That config file controls how an LLM generates OASIS assessment values. The optimizer runs the pipeline,

measures accuracy against gold-standard outputs, uses an LLM to propose better config text, and repeats until accuracy plateaus or a maximum number of rounds is reached.

1.2 The Group Config File — The Artefact Being Optimised

This is a JSON array where each element describes one logical group of related OASIS items. It is the only file the optimizer ever modifies. Copper provides the initial version. The partner team's system improves it and hands a new version back.

Schema of one group element:

```
// One element in group_configs.json
{
  "group_id":   "functional_status",       // unique name for this group
  "tab_id":     6,                          // OASIS section number
  "item_codes": ["GG0130A1","GG0130B1",..], // OASIS items this group handles

  // ── THE THREE OPTIMISABLE FIELDS ──────────────────────────
  "rag_query":            "...",  // query sent to vector store for this group
  "group_guidance":       "...",  // static CMS coding rules injected into every
call
  "llm_prompt_template":  "...",  // full prompt; must contain {rag_context},
                                  // {group_guidance}, {clinical_summary}
placeholders

  // ── NEVER MODIFIED BY THE OPTIMIZER ───────────────────────
  "skip_logic":           { ... }, // deterministic CMS skip-pattern rules
  "confidence_threshold": 0.65     // float; controls gap-question triggering
}
```

Field	Purpose	Optimizer may change?
rag_query	Query string used to retrieve relevant patient chunks from the vector store	YES — rewrite to retrieve more clinically relevant evidence
group_guidance	Static CMS coding rules injected verbatim into every LLM call for this group	YES — extend or clarify rules based on observed failures
llm_prompt_template	Full LLM prompt structure with placeholders	YES — rewrite structure and instructions; placeholders must be preserved
skip_logic	Deterministic skip-pattern rules	NEVER — clinical compliance boundary
confidence_threshold	Float threshold; controls when gap questions are generated	NEVER

SECTION 2

Input Dataset Format

Labeled patient records that drive both evaluation and optimization

2.1 Per-Patient Record

Each labeled patient is a directory. The system must handle every combination of optional files gracefully — warn when optional files are absent, fail fast when mandatory files are missing.

```
patient_XXXX/
  referral_packet.txt         MANDATORY  — hospital discharge referral (plain
text)
  tap_tap_gap_answers.json    MANDATORY  — nurse gap answers in structured format
  medication_list.json        MANDATORY  — 3-layer unreconciled medication list
  oasis_gold_standard.json    MANDATORY  — correct OASIS output for evaluation
  ambient_scribe.txt          OPTIONAL   — nurse SOC visit transcript (~70%
present)
  clinical_note.txt           OPTIONAL   — prior facility / visit note (~50%
present)
```

2.2 tap_tap_gap_answers.json — Format Copper Will Provide

A structured JSON object containing the nurse's answers to gap questions. The key field for matching is field_codes — always match by item code, never by question text.

```
{
  "sections": [
    {
      "section": "Functional",
      "questions": [
        {
          "id":           "gg0130_bathing",
          "title":        "Rate patient usual bathing performance",
          "type":         "radio",
          "field_codes": ["GG0130A1"],    // <- match on this, not question text
          "answer":       "03",           // the nurse's answer value
          "why_gap":      "Requires in-person nurse observation"
        }
      ]
    }
  ],
  "fields_auto_extracted": {
    "from_referral":    ["M0100","M1021"],
    "from_scribe":      ["GG0170I1"],
    "from_medications": ["N0415"]
  }
}
```

2.3 oasis_gold_standard.json — Format Copper Will Provide

A flat key-value object mapping every OASIS item code to its correct value. This is what the generated OASIS is scored against. The HIPPS code is included but scored separately — see Section 4.

```
{
  "M1021":    "I10",
  "M1023_1":  "E11.9",
  "GG0130A1": "03",
  "GG0130B1": "04",
  "M1311_1":  "0",
  "C0500":    "12",
  "D0150A1":  "1",
  "N0415E":   "1",
  "HIPPS":    "1AFLS",      // scored separately; see Section 4
  ...                       // all 120+ OASIS-E1 items
}
```

SECTION 3 The Pipeline — Steps 1 to 6

Build your own standalone OASIS generation flow

DESIGN PRINCIPLE — Config-Injectable Pipeline
The group config must be passed into the pipeline at runtime — never hard-coded.
The same patient must be runnable with different config variants without any code change.
This is what makes iterative optimization possible.

Step 1 — Load Patient Record

Read all files from the patient directory into a single record structure. Validate mandatory files on load.

```
FUNCTION load_patient_record(patient_directory):
  ASSERT gold_standard file exists, else RAISE error
  LOAD referral_text         from referral_packet.txt
  LOAD gap_answers           from tap_tap_gap_answers.json
  LOAD medication_list       from medication_list.json
  LOAD gold_standard         from oasis_gold_standard.json
  LOAD ambient_scribe        from ambient_scribe.txt    IF file exists ELSE null
  LOAD clinical_note         from clinical_note.txt     IF file exists ELSE null
  RETURN PatientRecord containing all of the above
```

Step 2 — Build Patient RAG

Construct an in-memory vector store for this patient from all available documents. Use one collection per patient so runs never bleed into each other. Delete the collection after the eval run completes.

Source type priority

Tag every chunk with a source_type when adding it to the store. The LLM prompt must instruct the model to prefer higher-priority sources when there is a conflict.

source_type	What it comes from	Priority in conflict
NURSE_ASSESSMENT	Nurse SOC transcript + gap answers	Highest — current direct observation
CLINICAL_NOTE	Prior facility or visit note	High — recent history
REFERRAL	Hospital discharge referral packet	Medium — background and orders
MEDICATIONS	Formatted medication list text	Medium — for medication-related items

Embedding spec

- Model: all-MiniLM-L6-v2, 384 dimensions, L2 distance — must match production exactly
- Chunk size: 512 tokens with 50-token overlap
- Collection naming: use a patient-scoped prefix (e.g. eval_{patient_id}) for easy bulk cleanup

```
FUNCTION build_patient_rag(patient_record):
  rag = new VectorStore(collection = 'eval_' + patient_record.id)
  rag.add(patient_record.referral_text,  source_type = REFERRAL)
  IF patient_record.ambient_scribe exists:
      rag.add(patient_record.ambient_scribe,  source_type = NURSE_ASSESSMENT)
  IF patient_record.clinical_note exists:
      rag.add(patient_record.clinical_note,   source_type = CLINICAL_NOTE)
  rag.add(format_medication_text(patient_record.medication_list),
          source_type = MEDICATIONS)
  RETURN rag

FUNCTION format_medication_text(medication_list):
  // Flatten the 3-layer list into a plain paragraph
  // Include: name, dose, route, frequency, high-risk flag
  // Note any discrepancies between layers (missing at home, wrong dose, OTC
extras)
  RETURN formatted_text
```

Gap answer re-injection (used in Step 5)

After gap answers are collected, add them back into the RAG before re-running affected groups. This gives the second pass direct access to the nurse's answers as retrievable evidence.

```
FUNCTION reinject_gap_answers(rag, answered_gap_questions):
  narrative = format answered questions as structured clinical paragraph
              e.g. 'Nurse confirmed GG0130A1 (Bathing) = 03 (Partial assistance)'
  rag.add(narrative, source_type = NURSE_ASSESSMENT)
  // NURSE_ASSESSMENT has highest retrieval priority —
  // subsequent rag.query() calls will surface this content first
```

Step 3 — Assemble Clinical Summary

Build the clinical_summary string that is injected into every group prompt. This is assembled once per patient — it is not a RAG retrieval. Construct it in priority order:

```
FUNCTION assemble_clinical_summary(patient_record):
  summary = []

  IF ambient_scribe exists:
    APPEND 'NURSE SOC ASSESSMENT:' + first 3000 chars of ambient_scribe

  answered_gaps = extract already-answered fields from gap_answers
  IF answered_gaps is not empty:
    APPEND 'NURSE GAP ANSWERS:' + format as clinical narrative

  IF clinical_note exists:
    APPEND 'PRIOR CLINICAL NOTE:' + first 1500 chars

  APPEND 'REFERRAL SUMMARY:' + first 1500 chars of referral_text

  RETURN all parts joined with double newline

  // IMPORTANT: When injecting into the LLM prompt, include the instruction:
  // 'If referral and nurse assessment conflict, always prioritise the nurse
  //  assessment — it reflects the patient's current status at Start of Care.'
```

Step 4 — Run OASIS Group Pipeline

The core LLM loop. For each group in the provided config: retrieve patient context via the group's rag_query, assemble the full prompt, call Claude Sonnet via AWS Bedrock, and parse the response into structured item results.

RAG query strategy

The rag_query field in each group config is the search string sent to the vector store. Below are the query patterns Copper currently uses — the partner team should use these as the starting point in the initial config and allow the optimizer to improve them:

OASIS Item Family	Starting RAG Query Pattern
Primary & secondary diagnoses (M1021, M1023)	patient diagnosis ICD-10 code medical conditions history
Functional / ADL / Mobility (GG0130, GG0170, M1800–M1860)	patient functional status ADL mobility transfer ambulation level of assistance
Skin and wounds (M1300–M1340)	skin wound pressure injury stage measurement drainage tissue treatment

Medications and high-risk drugs (N0415, M2001)	medication list anticoagulant opioid insulin digoxin high-risk drug
Cognitive status / BIMS (C0200–C0500)	cognitive status memory orientation word recall daily decisions
Mood / depression (D0150–D0160)	depression mood PHQ anxiety social isolation feeling down
Vitals and height / weight (M1060)	vital signs blood pressure heart rate oxygen saturation weight height
Continence (M1600–M1620)	urinary bowel continence catheter incontinence UTI
Special treatments (O0110, O0350)	IV infusion oxygen ventilator wound therapy COVID vaccination
Administrative / dates (M0090, M0102)	referral date start of care discharge date admission

Group pipeline logic

```
FUNCTION run_one_group(rag, clinical_summary, group_config, bedrock_client):
  // 1. Retrieve relevant patient context
  chunks = rag.query(group_config.rag_query, top_k = 5)
  rag_context = format chunks into text, prioritising NURSE_ASSESSMENT source type

  // 2. Assemble the prompt
  prompt = group_config.llm_prompt_template
           with {rag_context}       replaced by rag_context
           with {group_guidance}    replaced by group_config.group_guidance
           with {clinical_summary}  replaced by clinical_summary

  // 3. Call Bedrock
  raw_response = bedrock_client.invoke(model = Claude Sonnet, prompt = prompt)

  // 4. Parse response into item results
  RETURN parse_group_result(raw_response, group_config.group_id,
group_config.item_codes)


FUNCTION run_all_groups(rag, clinical_summary, group_configs, bedrock_client):
  RETURN [run_one_group(rag, clinical_summary, gc, bedrock_client)
          FOR EACH gc IN group_configs]
```

Expected LLM response format per group

The llm_prompt_template must instruct the model to return ONLY a JSON array with no preamble or markdown fences:

```
// LLM must return this JSON array for each group call:
[
  {
    "item_code":    "GG0130A1",
```

```
    "value":         "03",
    "confidence":   0.85,
    "justification":"Nurse SOC note documents patient required moderate assistance
                     bathing due to dyspnoea on exertion.",
    "source_cited": "ambient_scribe — paragraph 4"
  },
  ...
]
```

Bedrock call constraints

- Max 4096 output tokens per group call
- Max 10–12 parallel Bedrock calls at a time to avoid rate-limit errors
- Confirm exact model ID with Copper at kickoff — do not assume a model string

Step 5 — Generate Gap Questions and Answer Them

After the first-pass group results, identify items that need clarification. In the eval harness, gap questions are answered immediately from the record's tap_tap_gap_answers.json — no human is in the loop.

When to flag an item as a gap

- Confidence score is below the group's confidence_threshold (typically 0.65)
- Value is null or blank in the group result
- Item is in the always-gap list: BIMS (C0200–C0500), PHQ-2/9 (D0150–D0160), all GG discharge goals, wound measurements — these require in-person administration and can never be auto-extracted from documents

Gap question bundling rules

Questions must be grouped by clinical context to reduce nurse cognitive load in the real system. Apply these rules when generating gap questions:

Bundle	Items	Rule
Vitals	M1060, BP, HR, RR, O2	One question covers all vitals together
Diagnoses	M1021, M1023	One question covers primary + other diagnoses together
Medications	N0415, M2001, M2003	One question covers full medication review
BIMS interview	C0200, C0300, C0400, C0500	Full word-recall sequence — never bundle with other items
PHQ screening	D0150A–D0160	Emit PHQ-2 first; expand to full PHQ-9 only if PHQ-2 score >= 3
GG functional items	GG0130_*, GG0170_*	One question per GG item code — do not batch mobility items together

Wound detail	M1311, M1324	One question per wound site for dimensions and tissue type

Gap answering in the eval harness

```
FUNCTION answer_gaps_from_record(gap_questions, patient_record):
  // Build a lookup: {item_code -> answer_value} from the gap_answers file
  answer_map = {}
  FOR EACH section IN patient_record.gap_answers.sections:
    FOR EACH question IN section.questions:
      FOR EACH code IN question.field_codes:
        answer_map[code] = question.answer

  FOR EACH gap_question IN gap_questions:
    gap_question.answer = answer_map.get(gap_question.item_code)
    IF answer is null:
      LOG warning: 'No gap answer found for {item_code} in patient {patient_id}'
      // This item will be EXCLUDED from accuracy scoring — not counted as wrong

  // Re-inject answered gaps into RAG before re-running affected groups
  reinject_gap_answers(rag, answered_gap_questions)

  // Re-run only groups that had flagged items
  affected_groups = group_configs WHERE any item_code is in gap_questions
  updated_results = run_all_groups(rag, clinical_summary, affected_groups,
bedrock_client)

  RETURN updated_results
```

Step 6 — Build Final OASIS Document

Merge all group results (first pass + re-run gap groups) into one flat OASIS document. Apply skip-logic validation. The output must be structurally identical to oasis_gold_standard.json so the delta comparison works directly.

```
FUNCTION build_final_oasis(all_group_results, group_configs):
  // 1. Flatten all item results into a single {item_code: value} dict
  oasis_doc = {}
  FOR EACH group_result IN all_group_results:
    FOR EACH item IN group_result.items:
      oasis_doc[item.item_code] = item.value

  // 2. Apply CMS skip-logic from each group_config.skip_logic
  //    Examples of required skip rules:
  //    - If BIMS total (C0500) == '00' → null out C0300 and C0400
  //    - If PHQ-2 score (D0150A1 + D0150B1) < 3 → null out all PHQ-9 items
  //    - Group-specific conditions are defined in skip_logic field of each group
config
  oasis_doc = apply_skip_logic(oasis_doc, group_configs)

  // 3. Return flat dict — same schema as oasis_gold_standard.json
  RETURN oasis_doc
```

```
// NOTE: HIPPS code computation requires the CMS PDGM grouper.
// Copper runs HIPPS validation on their own infrastructure.
// Exclude 'HIPPS' from partner-side scoring unless grouper is provided.
```

SECTION 4

Delta Computation

Scoring the generated OASIS against the gold standard

4.1 What to Compute

For each patient run, compare every item in the generated OASIS against the gold standard. Produce three levels of score: item-level (correct / wrong), group-level (% correct within a group), and document-level (overall % correct for the patient).

```
FUNCTION compute_delta(generated_oasis, gold_standard, group_results,
unanswered_gap_codes):
  item_deltas = []

  FOR EACH item_code IN gold_standard:
    SKIP if item_code == 'HIPPS'               // scored separately
    SKIP if item_code IN unanswered_gap_codes  // excluded from denominator
    SKIP if item_code not in gold_standard     // not applicable for this patient

    generated_value = generated_oasis.get(item_code, MISSING)
    gold_value      = gold_standard[item_code]
    confidence      = get_confidence_for_item(item_code, group_results)

    is_correct   = (generated_value == gold_value)
    skip_error   = detect_skip_mismatch(item_code, generated_value, gold_value)
    overconfident = (confidence >= 0.85 AND is_correct == false)

    item_deltas.APPEND { item_code, generated_value, gold_value,
                         is_correct, skip_error, overconfident, group_id }

  group_accuracy = compute per-group accuracy from item_deltas
  doc_accuracy   = count(is_correct) / count(item_deltas)
  RETURN PatientDelta { patient_id, item_deltas, group_accuracy, doc_accuracy }
```

4.2 Scoring Rules

Scenario	Scoring Rule	Score
Generated value equals gold value	Exact string match on coded response ('03' == '03')	CORRECT
Skip applied correctly	Item should be skipped per CMS rules AND system skips it	CORRECT
Skip applied when it should not be	System skips but gold has a real value	WRONG + skip_error flag

Item scored when it should be skipped	System provides a value but gold says skip	WRONG + skip_error flag
Wrong coded value	Different value, no skip involved	WRONG
Item missing from generated doc	System produced no value for a scorable item	WRONG
No gap answer available for item	tap_tap_gap_answers had no match for this code	EXCLUDED from denominator — not counted as wrong
Item absent from gold standard	Gold standard omits the item entirely	EXCLUDED from denominator

4.3 Aggregate Scoring Across the Dataset

```
FUNCTION aggregate_scores(all_patient_deltas):
  FOR EACH group_id:
    group_accuracy[group_id] = mean of that group's accuracy across all patients
  overall_accuracy = mean of all patients' doc_accuracy
  RETURN { group_accuracy, overall_accuracy }

  // Also surface: skip_error_count per group (used as optimizer acceptance guard)
  // Also surface: overconfident_error_count per group (high optimizer priority)
```

SECTION 5 The Config Optimizer — Step 7

Iterative LLM-driven config improvement

5.1 Main Loop

```
FUNCTION run_optimizer(dataset, initial_config, bedrock_client,
                       n_rounds=10, accuracy_threshold=0.80,
                       min_improvement_pct=1.0, train_ratio=0.80):

  // Deterministic split — same seed every run
  train_set, holdout_set = split(dataset, train_ratio, seed=42)
  current_config = copy of initial_config
  stagnant_rounds = 0

  // Establish baseline
  baseline = eval_on_dataset(train_set, current_config, bedrock_client)
  LOG 'BASELINE overall = {baseline.overall}'
  SAVE checkpoint(round=0, config=current_config, scores=baseline)

  FOR round = 1 TO n_rounds:
    underperforming = groups WHERE baseline[group_id] < accuracy_threshold
    IF underperforming is empty:
      LOG 'All groups above threshold — stopping'; BREAK

    candidates = generate_candidates(current_config, underperforming,
```

```
                                    train_set, baseline, bedrock_client)
    best_config, best_scores = select_best_candidate(candidates, train_set,
                                                      baseline, bedrock_client)

    improvement = best_scores.overall - baseline.overall
    IF best_config is null OR improvement < min_improvement_pct / 100:
      stagnant_rounds += 1
      LOG 'Round {round}: no improvement. Stagnant = {stagnant_rounds}'
    ELSE:
      stagnant_rounds = 0
      current_config = best_config
      baseline = best_scores
      LOG 'Round {round}: promoted. Overall +{improvement}'

    SAVE checkpoint(round, current_config, baseline)

    IF stagnant_rounds >= 3:
      LOG 'Stagnant for 3 consecutive rounds — stopping'; BREAK

  // Final holdout validation
  holdout_scores = eval_on_dataset(holdout_set, current_config, bedrock_client)
  LOG 'HOLDOUT overall = {holdout_scores.overall}'
  WRITE final_config.json, optimization_history.json
  RETURN final_config, optimization_history
```

5.2 eval_on_dataset — Harness as Eval Engine

This function runs the complete Steps 1–6 pipeline on every patient in the dataset using a given config, and returns aggregate accuracy scores. It is called many times — once per candidate config evaluation — so performance matters.

```
FUNCTION eval_on_dataset(dataset, config, bedrock_client):
  all_deltas = []
  FOR EACH patient_record IN dataset:
    rag             = build_patient_rag(patient_record)
    clinical_summary = assemble_clinical_summary(patient_record)
    first_pass      = run_all_groups(rag, clinical_summary, config,
bedrock_client)
    gap_questions   = generate_gap_questions(first_pass, config)
    answered_gaps   = answer_gaps_from_record(gap_questions, patient_record)
    reinject_gap_answers(rag, answered_gaps)

    affected_groups = groups WHERE any item_code appears in answered_gaps
    IF affected_groups is not empty:
      updated       = run_all_groups(rag, clinical_summary,
                                     affected_groups, bedrock_client)
      final_results = merge first_pass with updated (updated takes precedence)
    ELSE:
      final_results = first_pass

    generated_oasis  = build_final_oasis(final_results, config)
    unanswered_codes = item_codes WHERE gap_question had no answer in record
    delta            = compute_delta(generated_oasis,
patient_record.gold_standard,
                                     final_results, unanswered_codes)
    all_deltas.APPEND(delta)
```

```
    DELETE rag collection for this patient  // cleanup — critical

  RETURN aggregate_scores(all_deltas)
```

5.3 generate_candidates — The Meta-LLM Call

For each underperforming group, select the most informative failure examples, build a structured optimization prompt, call Claude Sonnet, and receive 3 improved config variants per group.

Failure example selection — priority order

Failure Type	Priority	What to do
skip_error — skip applied incorrectly	HIGHEST — always include if present	Structural guidance problem; LLM can fix with explicit skip-condition language
overconfident + wrong (confidence >= 0.85, is_correct = false)	HIGH — include 1–2	Dangerous: system confidently wrong. Likely needs prompt restructure.
Same item wrong across 3+ patients systematically	HIGH — sample 2 examples	Config gap, not noise. High optimizer ROI.
Answer was in the document but not extracted	MEDIUM — include 1	Likely RAG query miss — optimizer should focus on rag_query for these.
Single-patient anomaly	LOW — skip in early rounds	May be data quality, not config. Adds noise to optimizer.

```
FUNCTION generate_candidates(current_config, underperforming_groups,
                             train_set, current_scores, bedrock_client,
                             n_candidates=3):
  all_candidates = []
  FOR EACH group IN underperforming_groups:
    group_config     = get group element from current_config by group_id
    failure_examples = select top-5 failures for this group from eval results
                       using the priority order in the table above
    meta_prompt      = build_meta_prompt(group_config, current_scores[group_id],
                                         failure_examples, n_candidates)
    llm_response     = bedrock_client.invoke(Claude Sonnet, meta_prompt)
    parsed_candidates = parse JSON array from llm_response  // n_candidates dicts
    all_candidates.APPEND (group_id, parsed_candidates)

  // Combine per-group candidates into complete config variants
  // Each variant = current_config with one group's fields replaced
  RETURN build_config_variants(current_config, all_candidates)
```

Meta-prompt structure

The prompt sent to the meta-LLM must include all of the following in order:

```
SYSTEM (static):
  You are an expert OASIS-E1 clinical coding specialist and LLM prompt engineer.
  Improve the configuration for a group of OASIS items so that an LLM extracts
  correct values from patient documents more accurately.
  Respond ONLY with a valid JSON array. No preamble. No markdown fences.

USER (assembled per group per round):

  GROUP: {group_id}
  OASIS ITEMS: {item_codes}
  CURRENT ACCURACY: {accuracy}% on {n_patients} training patients

  CURRENT CONFIG:
    rag_query:            {current rag_query}
    group_guidance:       {current group_guidance}
    llm_prompt_template:  {current llm_prompt_template}

  TOP FAILURE EXAMPLES (up to 5):
    For each failure:
      - item_code | generated value | gold value | confidence
      - failure_type: skip_error / overconfident / systematic / rag_miss
      - document excerpt that contained the correct answer (if found)

  TASK:
    Generate {n_candidates} improved configs for this group.
    For each candidate:
      1. Only change what is needed to fix the failure patterns shown above
      2. rag_query: rewrite to better retrieve the excerpts shown above
      3. group_guidance: add or clarify the rule that the failures violated
      4. llm_prompt_template: fix the instruction that produced the wrong output
      5. Preserve {rag_context} {group_guidance} {clinical_summary} placeholders
exactly

  RESPOND AS:
  [{ "rag_query":"...", "group_guidance":"...", "llm_prompt_template":"...",
"rationale":"..." }, ...]
```

5.4 select_best_candidate

```
ACCEPTANCE CRITERIA:
  - overall accuracy must improve by >= 3% over current baseline
  - no group may regress vs current baseline
  - skip_error count must not increase vs current baseline

FUNCTION select_best_candidate(candidate_configs, train_set, baseline,
bedrock_client):
  accepted = []
  FOR EACH candidate IN candidate_configs:
    scores = eval_on_dataset(train_set, candidate, bedrock_client)
    IF scores passes all ACCEPTANCE CRITERIA vs baseline:
      accepted.APPEND (candidate, scores)
```

```
  IF accepted is empty:
    RETURN (null, null)  // no improvement this round

  RETURN the (candidate, scores) pair with highest overall accuracy
```

SECTION 6

Output Artefacts

Files written after each optimization run

6.1 Output Directory Structure

```
optimization_runs/
  run_{YYYYMMDD_HHMMSS}/
    final_config.json            // PRIMARY DELIVERABLE — hand this to Copper
    baseline_scores.json         // per-group accuracy before optimization
    final_train_scores.json      // per-group accuracy after optimization (train
set)
    holdout_scores.json          // validation on holdout set
    optimization_history.json    // round-by-round summary
    failure_analysis.json        // per-group failure patterns that drove changes
    checkpoints/
      config_round_01.json       // config snapshot after each round
      config_round_02.json
      ...
    per_patient_deltas/
      {patient_id}_delta.json    // item-level diffs for every evaluated patient
    rejected_candidates/
      round_{n}_candidate_{m}.json  // every rejected candidate + rejection reason
```

6.2 optimization_history.json Schema

```
{
  "run_id":                     "run_20260307_142200",
  "n_train_patients":           40,
  "n_holdout_patients":         10,
  "initial_overall_accuracy":   0.714,
  "final_train_accuracy":       0.891,
  "final_holdout_accuracy":     0.878,
  "n_rounds_completed":         7,
  "stop_reason":                "stagnant_3_rounds",
                                // or 'all_groups_above_threshold'
                                // or 'n_rounds_reached'
  "rounds": [
    {
      "round":                 1,
      "groups_targeted":      ["functional_status","integumentary"],
      "candidates_evaluated": 6,
      "best_delta":           0.041,
      "promoted":             true,
      "scores": { "functional_status":0.82, "overall":0.755 }
    },
    ...
```

```
  ]
}
```

6.3 final_config.json

Drop-in replacement for the original group_configs.json provided by Copper. Schema must be identical. Add a read-only _optimizer_meta block to each group element:

```
{
  "group_id":              "functional_status",
  "tab_id":                6,
  "item_codes":            ["GG0130A1","GG0130B1",...],
  "rag_query":             "...",    // may be changed
  "group_guidance":        "...",    // may be changed
  "llm_prompt_template":   "...",    // may be changed
  "skip_logic":            {...},    // UNCHANGED
  "confidence_threshold": 0.65,      // UNCHANGED
  "_optimizer_meta": {
    "original_accuracy":    0.68,
    "final_accuracy":       0.89,
    "rounds_to_converge":   4,
    "last_modified_round":  4,
    "run_id":               "run_20260307_142200"
  }
}
```

SECTION 7 Phased Delivery Plan

Milestones, acceptance gates, and timelines

Phase	Deliverable	Acceptance Gate	Weeks
Phase 0Setup	AWS Bedrock access confirmed. Dev environment working. Copper provides: group_configs.json, labeled dataset, gold standard format confirmation.	Bedrock call succeeds from local machine. One patient record loads without errors. Data handoff signed off by Copper.	1
Phase 1Harness	Steps 1–6 working on a single patient. Patient loader, RAG build, group pipeline, gap generation and answering, final OASIS document.	Harness runs on one labeled patient and produces an OASIS document that can be delta-compared against the gold standard without errors.	2–3
Phase 2Eval Engine	eval_on_dataset() running on full labeled set. Per-patient delta files. Aggregate accuracy report. Baseline scores produced.	50-patient eval run completes in under 15 minutes. Baseline per-group accuracy report reviewed and confirmed reasonable by Copper before optimizer starts.	3–4

Phase 3Optimizer v1	generate_candidates() + select_best_candidate(). Meta-LLM call with failure examples. Candidate evaluation. Acceptance criteria gate.	Optimizer generates candidates for one underperforming group. At least one candidate passes acceptance gate. Config diff is human-readable.	4–6
Phase 4Full Loop	run_optimizer() main loop with n_rounds, stagnation detection, holdout validation gate. All output files written per Section 6 spec.	Full 10-round loop completes on 50-patient dataset. final_config.json produced and schema-valid. Holdout accuracy within 2% of train accuracy.	6–8
Phase 5Handback	final_config.json, optimization_history.json, per_patient_deltas/ delivered to Copper. Code review by Manas. Debrief session.	Copper confirms final_config.json is drop-in compatible. Doc accuracy >= 85%. Per-group failure analysis reviewed by clinical lead.	9

SECTION 8 Constraints, Targets & Open Questions

8.1 Hard Constraints

- Build your own pipeline — do NOT import or call any Copper codebase module
- Only modify rag_query, group_guidance, llm_prompt_template — never touch skip_logic, confidence_threshold, item_codes, or any other config field
- Holdout set (20%) is sacred — never used during optimization rounds, only for final validation
- No PHI in output files — all patient references use patient_id codes, never real names or MRNs
- final_config.json must be schema-compatible with the input group_configs.json — Copper does the DB write
- Delete temp RAG collections after each patient eval run — do not let them accumulate

8.2 Acceptance Targets

Metric	Target	Notes
Overall OASIS item accuracy (train)	≥ 85%	Measured at end of full optimization run
Overall OASIS item accuracy (holdout)	Within 2% of train accuracy	Guard against config overfitting
No group regression	No group drops below (baseline − 2%)	Checked per candidate before promotion

No new skip errors	Skip error count must not increase vs baseline	Structural red flag if violated
Config schema valid	final_config.json validates against input schema	Run schema check before handback
Eval run time	50-patient eval run in under 15 minutes	Tune Bedrock concurrency if needed
Full loop time	10-round optimization loop in under 3 hours	Profile and optimise if exceeded

8.3 Open Questions — Resolve at Kickoff

Topic	Question	Who to ask
Bedrock model ID	What is the exact Claude Sonnet model ID currently in use? Do not assume.	Manas
Vector store	Is production on ChromaDB or has it migrated to OpenSearch? Implement what production uses.	Nikhil
Config file schema	Share the exact group_configs.json schema including any fields not shown in this doc.	Nikhil
Gold standard format	Confirm oasis_gold_standard.json schema and whether HIPPS is computed client-side or not.	Manas / Balaji
Dataset size at kickoff	How many labeled patients will be ready on Day 1? Phase 2 needs at least 30 for a meaningful baseline.	Balaji
Clinical review	Clinical lead must review and sign off the Phase 2 baseline report before Phase 3 starts.	Kathy

CONTACTS

CONTACTS
Manas (CTO) — architecture, Bedrock access, config schema, all blockers
Nikhil — pipeline implementation details, vector store config, group config schema
Balaji — data access, AWS permissions, PostgreSQL schema
Kathy (Clinical Lead) — OASIS coding rules, skip logic, clinical accuracy sign-off
Do not infer pipeline behaviour from documentation alone — ask when uncertain.

⚠️ Copper Digital Internal / Partner-Restricted. Do not distribute. March 2026.